PIXELATE

Fork https://github.com/vruga/pixelate.git and clone it.
Make changes locally, push them on your fork.
In each google form, attach the link to the repository along with the output codeword/ image as per question.

Problem Statement 1

Trial I

Lore

Dobby has been tasked by Hermione with applying two consecutive magical filters to a photograph of the Hogwarts grounds. Each filter is a 3×3 convolution kernel. Dobby is enthusiastic but easily tired. The image is enormous and he is already behind on his sock-knitting.

You have to help Dobby figure out how to apply both kernels to the image with **as little computation as possible**. Show your working and state the most efficient approach you can find.

The Two Kernels

Hermione's two filters are:

K_1:

```
 1   2   1
 0   0   0
-1  -2  -1
```

K_2:

```
 1   0  -1
 2   0  -2
 1   0  -1
```

Your Task

You need to convolve an image with both K_1 and K_2. Applying each 3×3 kernel naively costs **9 multiplications per pixel**. Find a way to achieve the combined effect with the fewest total multiplications per pixel. Show your working and state how many multiplications per pixel your approach requires.

> **Hint:** Separable convolutions.

Submission Google Form

https://forms.gle/BFRjVUB6o8GCNgmv8

Problem Statement 2

Trial II

Lore

Dumbledore's Pensieve was shattered during the Battle of Hogwarts. His most important memory — containing the location of a final safeguard — survived, but only in pieces. Three wizards each preserved one fragment before the Pensieve broke. To protect the fragments from falling into the wrong hands, each wizard applied a different concealment charm before storing their piece.

Neville has recovered all three fragments from their hiding places. He needs you to undo the charms and reconstruct the original memory.

The Three Fragments

Dumbledore used **additive secret sharing** to split the memory into three shares before the charm was applied:

```
share1 + share2 + share3 ≡ memory  (mod 256)
```

Each share on its own is indistinguishable from random noise. The memory can only be recovered by reversing each charm and summing all three shares.

You are given three grayscale images:

File	Contains	Charm Applied
`fragment_a.png`	**Share 1**	Horizontally flipped, then inverted (255 – value)
`fragment_b.png`	**Share 2**	No charm — stored as-is
`fragment_c.png`	**Share 3**	Vertically flipped

Your Task

Write a C++ program that reverses the charm on each fragment, reconstructs the memory using the additive formula above, and saves it as `memory_restored.png`. Print the number of non-zero pixels in the result as a sanity check.

Expected output:

```
Memory restored. Non-zero pixels: <count>
Saved to: memory_restored.png
```

Open `memory_restored.png` — the reconstructed memory will show a visible message. Write down what it says and submit it alongside your code.

Submission Google Form

https://forms.gle/fLEMd2T36DEFNJn59